Meter report: mapping rules

Input and output structures are in the attached schema files (readings.schema.json, report.schema.json). Notation: source.* is the input document, report.* the output. Array indices start at 0. `:=` assigns; [cond] filters an array; `where` is the same as [ ] and may be written either way.

1. Identification

```
report.meter       := upper(source.readings.meterId)
report.site        := source.readings.site.code + " / " + source.readings.site.name
report.unit        := source.readings.unit  -- "kWh" if empty  (1)
```

(1) "Empty" means absent, null or an empty string.

2. Period

```
report.periodStart := min(source.readings.samples[*].ts)
report.periodEnd   := max(source.readings.samples[*].ts)   -- both over ALL samples,
whatever the quality (2)
```

(2) Timestamps are ISO 8601 in UTC, so they sort as text. Without samples the two fields are not sent.

3. Quantities

Only samples with quality "OK" take part, unless stated otherwise.

```
good := source.readings.samples[quality == "OK"]
report.count         := count(good)
report.total         := round(sum(good.value), 3)
report.average       = round(report.total / report.count, 2)  -- (3)
report.maxValue      := max(good.value)                       -- (3)
report.qualityIssues := count(source.readings.samples where quality != "OK")
report.firstSample   := source.readings.samples[0].value      -- (4)
```

(3) Not sent when good is empty.

(4) The first element of the array as received, regardless of its quality; not sent when there are no samples.

Rounding: half up. `sum` of nothing is 0.

4. Open points

TBC with the metering team: whether "EST" (estimated) samples should count as good. For now they do not.